The `get_qtarget_ddqn` function just computes $Q_{new} = r_t + \gamma \times x$:

```
>>> def get_qtarget_ddqn(qvals,r,df,done):
>>>     targets = r + (1-done) * df * qvals
>>>     return targets
```

We provide `done`, which is a Boolean, because if the episode of the game is done, there is no next state on which to compute $Q(s_{t+1})$, so we just train on r_t and set the rest of the equation to 0.

That's all there is to double Q-learning; just another simple way to improve training stability and performance.

10.6. Training and attention visualization

We have most of the pieces now, but we need a few other helper functions before training.

Listing 10.8. Preprocessing functions

```
import gym
from gym_minigrid.minigrid import *
from gym_minigrid.wrappers import FullyObsWrapper, ImgObsWrapper
from skimage.transform import resize

def prepare_state(x):                          1
    ns =
torch.from_numpy(x).float().permute(2,0,1).unsqueeze(dim=0)#
    maxv = ns.flatten().max()
    ns = ns / maxv
    return ns

def get_minibatch(replay,size):                2
    batch_ids = np.random.randint(0,len(replay),size)
    batch = [replay[x] for x in batch_ids] #list of tuples
    state_batch = torch.cat([s for (s,a,r,s2,d) in batch],)
    action_batch = torch.Tensor([a for (s,a,r,s2,d) in
batch]).long()
    reward_batch = torch.Tensor([r for (s,a,r,s2,d) in batch])
    state2_batch = torch.cat([s2 for (s,a,r,s2,d) in batch],dim=0)
    done_batch = torch.Tensor([d for (s,a,r,s2,d) in batch])
    return state_batch,action_batch,reward_batch,state2_batch,
done_batch
```